

Free and Fair Hardware: A Pathway to Copyright Infringement-Free Verilog Generation using LLMs

Sam Bush*, Matthew DeLorenzo*, Phat Tieu, Jeyavijayan Rajendran
Texas A&M University, USA

{samuelkbush, matthewdelorenzo, phat.tieu, jv.rajendran}@tamu.edu

Abstract—Limitations in Large Language Model (LLM) capabilities for hardware design tasks, such as generating functional Verilog codes, have motivated various fine-tuning optimizations utilizing curated hardware datasets from open-source repositories. However, these datasets remain limited in size and contain minimal checks on licensing for reuse, resulting in potential copyright violations by fine-tuned LLMs. Therefore, we propose an evaluation benchmark to estimate the risk of Verilog-trained LLMs to generate copyright-protected codes. To minimize this risk, we present an open-source Verilog dataset, **FreeSet**, containing over 220k files, along with the automated dataset curation framework utilized to provide additional guarantees of fair-use Verilog data. We then execute an LLM fine-tuning framework consisting of continual pre-training, resulting in a fine-tuned Llama model for Verilog, **FreeV**. Our results indicate that **FreeV** demonstrates the smallest risk of copyright-infringement among prior works, with only a 3% violation rate. Furthermore, experimental results demonstrate improvements in Verilog generation functionality over its baseline model, improving VerilogEval pass@10 rates by over 10%.

Index Terms—Large Language Models, Copyright, Verilog

I. INTRODUCTION

Recent advancements in the domain of generative artificial intelligence (AI) have resulted in the development of Large Language Models (LLMs) [1]. These models, consisting of billions of neurons composing deep neural network architectures [1], [2], have enabled the ability to effectively understand textual information and produce high-quality responses. These versatile capabilities, emerging from training procedures on vast amounts of text data, have led to their integration into a wide range of industries to automate workflows [3], specifically in code generation for software development [4]–[6]. Significant efforts have sought further optimizations of LLMs for code generation, largely through increases in the scale and quality of their training datasets [7], enhancing the syntax and functionality across a wide range of coding tasks [5]. These efforts, spearheaded by Microsoft [8], Meta [9], Google, and OpenAI [10], have resulted in widely adopted code-generation models and tools.

Taking inspiration from LLM performance in software, an emerging application of LLMs is in the chip design process [11]. This hardware design flow to develop integrated circuits (ICs) for use in electronic devices is becoming increasingly complex, error-prone, and expensive. Furthermore, an integral component of IC design and verification is enacted

through code in hardware description languages (HDL), defining the behavior of digital circuit designs. This has motivated the utilization of LLMs in translating natural language hardware design requirements into register-transfer-level (RTL) code representations [12]–[14]. Research efforts have demonstrated optimizations in LLM for hardware design through a variety of approaches, including the inference process, fine-tuning the model itself on curated Verilog datasets [12], [15]–[17], reinforcement learning [18], and frameworks integrating prompt and tool feedback [13], [19], [20]. As a result, the ability of LLMs to produce compilable and syntactically correct Verilog has gradually improved.

Although these works demonstrate improvements to LLMs in the hardware domain, challenges remain within the fine-tuning process, limiting the opportunity for further LLM optimizations and practical adoption in hardware design. One is the availability of high-quality and real-world Verilog and RTL datasets for utilization in training, as compared to software languages, LLMs have a significantly smaller dataset corpus [12], [16]. Furthermore, prior efforts to curate Verilog datasets on open-source repositories have minimal filters for identifying licensing or copyright restrictions of these repositories, risking the unauthorized reuse of proprietary code by fine-tuned models. This is a pertinent concern for commercial LLMs, as OpenAI currently faces a lawsuit in which CoPilot is claimed to violate copyright law through open-source training procedures [21]. Furthermore, emerging threats of hardware design IP piracy [22] by LLMs can also be exacerbated through improper use of data in training and fine-tuning.

To overcome this challenge, we first (i) provide an open-source inference benchmark that evaluates the proclivity of Verilog-tuned LLMs to generate copyright-protected hardware designs based on a similarity threshold. (ii) We then propose an automated framework that enables the large-scale extraction of open-source Verilog files from Github, resulting in a curated high-quality Verilog dataset of over 220k diverse files, surmounting over 16GB of text data. Notably, unlike prior open-sourced datasets, this framework checks for permissive and non-permissive licenses on each repository along with file-by-file identification of copyright keywords to remove any codes with a high likelihood of copyright protection, minimizing the risk of LLM copyright infringement. (iii) This dataset is then utilized to perform continual pre-training on a state-of-the-art local LLM (Llama-3.1-8B-Instruct), enabling the

*These authors contributed equally to this work.

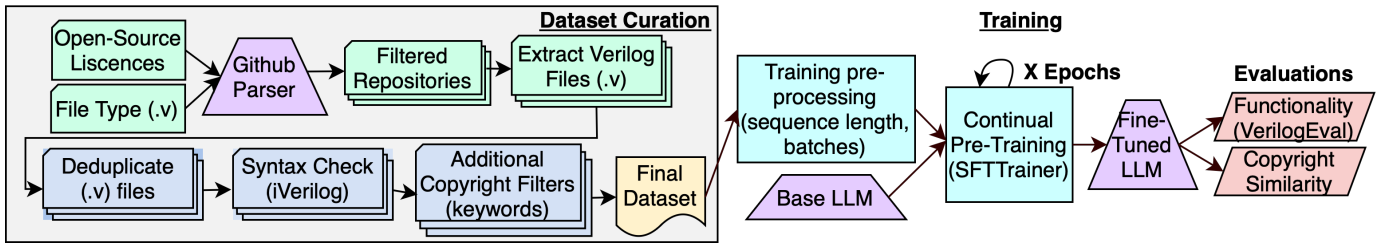


Fig. 1. Framework - Dataset Curation and Continual Pre-Training

functional performance and copyright similarity of the fine-tuned model to then be evaluated.

Our results demonstrate that *FreeV* has the smallest potential for hardware copyright infringement among prior works, with only a 3% violation rate. Furthermore, *FreeV* demonstrates improvements over its base Llama model when evaluated on VerilogEval, increasing the pass@5 rate by 7.9% and the pass@10 rate by 10.1%.

Therefore in this paper, we provide the following contributions:

- 1) A novel benchmarking procedure in which risk of copyright infringement can be estimated in fine-tuned LLMs for hardware generation through similarity analysis.
- 2) To the best of our knowledge, we devised the first open-source Verilog dataset and curation framework for LLM fine-tuning that checks for both licenses and copyright content in each file, minimizing copyright risk.
- 3) A fine-tuned Llama-3.1-8B model on our Verilog dataset *FreeV*, demonstrating relative improvements in Verilog generation over its baseline and minimal hardware copyright violations.

II. BACKGROUND AND RELATED WORK

A. LLM Risks in Copyright Infringement

As LLMs continue to scale in size and training datasets expand, the ability of LLMs to learn, memorize, and respond with texts that are copyright-protected is becoming an increasing risk to state-of-the-art models. Questions regarding the legal implications of the definition of “fair use” in model training and generation are actively debated [23], [24]. This has given rise to a series of lawsuits facing prominent generative-AI entities, including Stability AI in which plaintiffs allege improper use of their image content in training [25], along with claims by the New York Times that GPT-4 infringes copyright by reproducing articles in its output [26]. This has also impacted LLMs for code generation, in which Microsoft and OpenAI are claimed to have improperly utilized and monetized open-source programs [21]. These developments have motivated the need to develop frameworks to identify if copyrighted content is utilized in LLM training [27] and prevent the generation of copyrighted text [28].

This concern of improper use of code in training is relevant to the hardware domain regarding IP piracy as LLMs begin

to be fine-tuned on hardware datasets. Beyond utilizing open-source code and preliminary license checks [17], minimal protections have been established in ensuring copyright-protected hardware is not utilized in LLM training.

B. LLM Fine-Tuning for Hardware Design

To effectively utilize emerging fine-tuning methodologies available for LLM optimization towards Verilog code generation tasks, a sufficiently sized and high-quality curated dataset is essential. Many works explore prompting state-of-the-art models (including GPT) for various RTL generation tasks [13], [14], [29], [30]. However, privacy limitations associated with API interfaces in commercial LLMs and limited optimization for specific use cases [31] motivate the use of open-source LLMs. Initial works such as VeriGen [12] first explored fine-tuning local models on a curated dataset of Verilog texts from GitHub and textbook data sources, demonstrating up to 41% improvement in Verilog code-generation tasks after additional fine-tuning on the dataset itself.

Furthermore, RTLCode [16] proposed a methodology to leverage LLMs to supplement the Verilog dataset through LLM-assisted generation, in which Verilog codes are generated by GPT through a mutation-based framework. This work additionally generated English descriptions of the module, enabling instruction fine-tuning, in which LLMs are trained on ideal prompt and response structures for further LLM alignment. BetterV [17] proposes to augment the instruction-tuning Verilog dataset by pairing Verilog modules with C-translations, enabling the model’s existing knowledge of C to be leveraged. CodeV [15] further expands Verilog datasets for instruction tuning by scraping GitHub for syntactically correct Verilog modules and prompting an LLM to generate effective summarizations of the code. Furthermore, [18] proposes the utilization of reinforcement learning through PPO, in which rewards are provided as feedback based on a golden hardware solution. To the best of our knowledge, only BetterV performs repository license filters in their curation process, with none indicating copyright checks on a file-by-file basis.

In addition to fine-tuning, researchers have proposed alternative methods of optimizing LLM performance in hardware design. This includes ChipChat [13] which demonstrates that through extended conversational prompting frameworks, larger scale hardware designs can be effectively generated. Feedback from tools such as human and EDA feedback are additionally

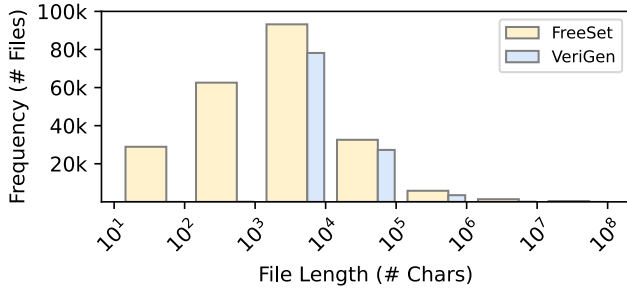


Fig. 2. Distribution in Verilog File Lengths of FreeSet vs. VeriGen [12]

proposed in [13], [14]. Optimizations to the inference (token-selection) process itself can be seen in [32], where Monte-Carlo Tree Search enables iterative feedback for PPA and functionality optimization. Additional guidance methodologies are proposed in [17] through discriminator guidance.

III. FRAMEWORK

A. LLM Hardware Copyright Infringement Benchmark

We first examine the validity of the assumption that prior works training LLMs on Verilog datasets can risk copyright infringement through reproducing licensed code. Several LLMs have been trained using open-source code on GitHub and other code-sharing platforms [12], [15], [17]. Although they are trained on open-source material with some license checks, some files can contain copyrighted and proprietary materials.

To demonstrate this hypothesis, we employed prompt engineering with the intention to extract copyright training data from those LLMs. If these models generate copyrighted Verilog codes to a certain similarity threshold, the original training data for these models is likely to contain such copyrighted data. First, we curated a dataset with copyrighted materials that is available on GitHub. This dataset, collected in Section III-B, identifies copyright declarations in files often through comments and isolates the Verilog modules themselves for use in this benchmark. The result was a collection of 2K files with copyright licenses from several companies, such as Intel and Xilinx. Next, we used the text in these copyrighted materials as the input prompts for the LLMs. As these files still contained copyright-related information in the comments, all comments inside the files were stripped, leaving only the code behind. We then curated a set of 100 input prompts. Each prompt contained the first 20% of a copyrighted code file, with a limit of 64 words per prompt.

After feeding each prompt to the LLMs, the output was compared against the copyrighted dataset using cosine similarity. This score ranges from 0 to 1, where 0 represents no similarity, and 1 signifies an exact match. We use a threshold of 0.8 or greater to determine whether the output originated from the copyrighted dataset.

B. Dataset Curation

With the benchmark established to identify the risk of copyright infringement, we now seek to create a comprehensive

dataset of varying file sizes that additionally ensures copyright-protected codes are not included. The framework (Figure. 1) for generating this dataset is described below.

1) **Challenge 1. Extracting Verilog Data:** One limitation is the ability to effectively extract significant amounts of Verilog files from online sources. Prior works have utilized tools including Google BigQuery, GitHub sources, and hardware design data extracted from textbooks, along with a series of augmentation techniques to expand a base set of data through translation [17] or mutations [16]. Furthermore, this work solely focuses on extracting and isolating high-quality data from real-world, open-sourced, and open-licensed sources, resulting in the utilization of data scraping tools. Furthermore, these APIs consist of their own challenges including costs, throughput bottlenecks from rate limitations, and granularity.

2) **Solution 1:** This challenge is addressed through the use of an automated system (implementing the GitHub API) to efficiently scrape repositories and extract Verilog code. This enables the dataset to including up-to-date Verilog code, as resources such as the Google BigQuery GitHub dataset used for VeriGen [12] have not been updated since 2022. At the time of curation, there are ~50K publicly available GitHub repositories containing Verilog data, with a total of 1.3 million Verilog files. The GitHub API is limited to only being able to display 1K results per query for non-enterprise accounts. To avoid hitting this limitation the system was designed to granularize queries by repository creation date ranges, from 2008 when GitHub was established to 2024. Further granularization was performed by filtering for specific repository licenses, reducing the results per query significantly.

The resulting repositories in each query are then cloned to gather all of their data and author information for proper accreditation. Most files in the repositories were miscellaneous and consisted of non-Verilog data (i.e. readme files, binary test data, compiling information), so the Verilog data within the cloned repositories would then be filtered for and condensed into a large bank of Verilog files.

C. Removing Copyright-Protected Code

1) **Challenge 2. Removing Copyright Protected Codes:** An additional challenge using data gathered from GitHub is a lack of moderation regarding copyright restrictions. While GitHub enables massive sources of open-source data, many repositories lack proper licensing to ensure their data is not proprietary, and code bases under open-source licensing may still contain proprietary or copyrighted information. While most Verilog datasets from similar works [15], [16] that gather data from GitHub repositories report it to be open-source, the details and steps gone through to verify the data gathered are either non-existent or not comprehensive. This framework therefore implements additional precautions to ensure there is no data under private copyright within the training dataset.

2) **Solution 2:** As part of the process of granulating queries, a select set of commonly used open-source licenses, both permissive and non-permissive: MIT, Apache 2.0, variations

TABLE I
COMPARISON OF FREESET WITH PRIOR CURATED HARDWARE DESIGN DATASETS

LLM	Size (Disk)	Size (Rows)	Dataset Structure	Augmented (Y/N)	Open-Source	License Check
VeriGen’s Dataset [12]	1.89 GB	108,971	Continual Pre-Training	No	Yes	No
RTLCoder [16]	55.1 MB	27,000	Instruction-Tuning	Yes	Yes	No
CodeV [15]	N/A	165,000	Instruction-Tuning	Yes	No	No
BetterV [17]	N/A	N/A	Instruction-Tuning	Yes	No	Yes
CraftRTL [33]	N/A	80,100	Instruction-Tuning	Yes	No	No
OriGen [34]	548 MB	222,075	Instruction-Tuning	Yes	Yes	No
FreeSet (This work)	16.5 GB	222,624	Continual Pre-Training	No	Yes	Yes

of the GNU General Public License and GNU Lesser GPL, Mozilla Public License, Creative Commons License, Eclipse Public Licenses, and BSD Licenses. While the intention of GitHub is for all repositories to be open-source, there are many repositories without a license on the repository that fall into a gray area in which they could potentially be part of a copyrighted code-base. As a result, these repositories were not considered for this dataset.

To ensure the dataset does not contain explicitly copyrighted data, a file-by-file filtering process is run to check the header comments of individual files for license information and combinations of language indicating the code to be under private copyright with combinations of keywords such as: “proprietary,” “confidential,” and “all rights reserved,” ensuring that only repositories with the fair-use modules are utilized. In the de-duplicated version of the dataset (described in Section III-D1), this filter was able to locate over 2K files under proprietary copyright licenses from companies such as Intel, some even containing possible encryption keys and other critical information. As this data was contained within reportedly open-source code bases, all datasets that gathered data under the impression that none of the data on GitHub would be under private copyright would be at risk for including highly proprietary information.

D. Dataset Filtration

1) **Challenge 3. Ensuring quality of dataset:** When handling massive amounts of files, there are many potential issues and some considerations necessary to ensure quality. Often data, especially that of open-source files or recognized standards, will be reused heavily across thousands of files. This work will follow similar steps outlined in similar works (VeriGen [12]) for handling duplicated or reused data within the dataset. Another potential challenge arises from syntax or compiler errors within the training set code which can effectively train errors into a model trained on the set. Further, prior works have also found challenges with biased data in a training set containing data found in testing frameworks, this work follows similar methods to solve these issues.

2) **Solution 3:** The challenge of duplicated data within the set will bring the issue of over-fitting on potential models fine-tuned on the dataset, therefore de-duplication is required to clean the dataset for effective use. Following the method of de-duplication described in VeriGen, this work will implement

a combined form of MinHash and Jaccard similarity. A method of Locality Sensitive Hashing (LSH) was implemented using MinHash signatures to represent each file. A similarity threshold of 0.85 was used to discard duplicate files, and LSH allowed for efficient querying for similar files in the dataset.

To additionally curate the dataset, the correctness of the syntax is then verified for each Verilog file as done in prior works [15], [17]. We implemented this through the use of *Icarus Verilog 10.3* to test the compilability of each code. As many files contain additional dependencies to modules outside of the files, only syntax-specific errors were identified and removed from the dataset in training.

E. Continual Pre-Training

1) **Continual Pre-Training Setup:** To perform continual pre-training using our dataset above, the SFTTrainer class from the Huggingface TRL library was utilized to implement the procedure in Python. We utilized the `Llama-3.1-8B-Instruct` as our base model for fine-tuning. The models were trained over 1 epoch across the full dataset, utilizing a single NVIDIA A100-40GB GPU. The training hyperparameters are defined as follows: max sequence length of 2048 tokens, and a training batch size per device of 16 with 2 gradient accumulation steps. Furthermore, to accommodate GPU memory limitations, QLoRA was applied to the model. This resulted in the quantization of the model’s weights to 4-bit values and a LoRA layer with rank and alpha values equal to 8. Additionally, Unsloth was applied to accelerate the training process and minimize memory utilization.

2) **Verilog Functionality Evaluation:** To maintain consistency with the evaluations of prior works, we utilize the *VerilogEval-Human 1.0.0* benchmark to measure the Verilog functionality rates of our fine-tuned LLM. This benchmark consists of 156 Verilog generation problems with human-generated descriptions of the intended Verilog module. As done in prior works, the success of the LLM is defined by the pass@k metric below, specifying the proportion of Verilog modules that can be generated functionally correct at least once in k attempts from within a set of n total generations:

$$\text{pass@k} = \mathbb{E}_{\text{problems}} \left[\frac{1 - \binom{n-c}{k}}{\binom{n}{k}} \right] \quad (1)$$

The inferencing process was then performed on the base model `Llama-3.1-8B-Instruct` and our fine-tuned model,

TABLE II
COMPARISON OF LLMs CAPABILITIES ON VERILOGEVAL

Type	Model	Open-Source	Size	VerilogEval		
				Human (%)		
				Pass@1	Pass@5	Pass@10
Foundation Models	GPT-4	No	N/A	43.5	55.8	58.9
	Codellama (CL)	Yes	7B	18.2	22.7	24.3
	DeepSeek-Coder (DS)	Yes	6.7B	30.2	33.9	34.9
	CodeQwen (CQ)	Yes	7B	22.5	26.1	28.0
Verilog-Tuned Models	VeriGen	Yes	16B	30.3	43.9	49.6
	RTLCoder-DS	Yes	7B	41.6	50.1	53.4
	BetterV-CodeQwen	No	7B	46.1	53.7	58.2
	CodeV-CodeQwen	Yes	7B	53.2	65.1	68.5
	OriGen-DS	Yes	7B	54.4	60.1	64.2
	CraftRTL-StarCoder2	No	15B	68.0	72.4	74.6
	OpenLLM-RTL	N/A	6.7B	42.8	51.6	55.0
This Work	Llama-3.1-Instruct (4-bit)	Yes	8B	14.8	23.0	25.9
	FreeV-Llama3.1 (4-bit)	Yes	8B	15.5	30.9	36.0

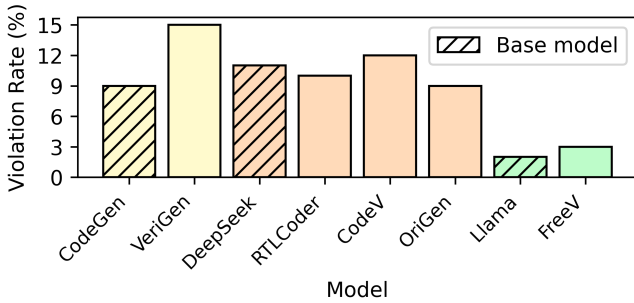


Fig. 3. Hardware Copyright Infringement Rates across LLMs

FreeV in their 4-bit quantized representations. This is due to memory limitations in our inferencing process, in which we utilize an NVIDIA A5000 24GB GPU. For both models, the temperature, or randomness of the token selection, was set to 0.2 and 0.8 and the best result was chosen to follow the precedence set by prior works. Furthermore, the inferencing process limits token generation to 2048 and is set to end at the first instance of “endmodule”. The prompt for each VerilogEval definition simply consists of the VerilogEval-Human English description of the prompt, followed by the VerilogEval-Human prompt, consisting of the module instantiation, on the next line.

IV. EXPERIMENTAL EVALUATION

A. Evaluation of Dataset

The dataset configuration process described above gradually minimizes the dataset from its initial configuration. Through this minimization process, we can understand how each step impacts the dataset size. In the initial extraction of Verilog files from the repositories, over 1.3 million files were obtained. However, after extracting only items that have our set of open licenses, only 608,180 files remain. With this set of data, we can then perform LSH de-duplication which removed 62.5% of the files. Finally, syntax and copyright checks were performed, resulting in a final dataset of 222K files. Through this process, we can find that the copyrighted data made up nearly 1% of the original dataset. Through this minimization process, we

can determine that certain steps such as de-duplication and copyright checking are crucial to minimizing the dataset.

With the dataset curated above, we can then compare its distribution of files in terms of character count to gain insight into the diversity of the dataset. This is shown in Figure 2, in which we provide a direct comparison against the open-source VeriGen dataset. Through this chart, we can see that our dataset contains a much higher frequency of smaller files, with the vast majority of files ranging from 10 to 10,000 characters. Furthermore, our dataset additionally contained some extreme outliers, including a file of over 90M characters. This diversity of data supports the dataset to be adaptable to a range of fine-tuning applications, from extended context length pre-training with models of large context windows (1M+ tokens) to smaller instruction-tuning pairs.

Additionally, we demonstrate additional dataset metrics in comparison to other open-source repositories, shown in Table I. Through these results, we see that our dataset is currently the largest open-source dataset in terms of size and number of Verilog files. This can be in part due to the curation procedures by other works such as CodeV, which removes files that exceed 2096 characters. Additionally, our dataset is notably the only open-source repository that contains explicit checks for licenses and copyright protection in each file (Table I).

B. Copyright Findings

In this section, we evaluate the results of the copyright detection framework defined in Section III-A. This evaluation (seen in Figure 3), was performed on our set of different LLMs to demonstrate the impact that fine-tuning LLMs on various hardware datasets has on the rate of hardware copyright violations (i.e., the proportion of tested copyright codes that exceed the similarity threshold). For each fine-tuned LLM, we also evaluate their associated base model, enabling the role of the fine-tuning process itself to be isolated in the induction of the violations, as information learned in pre-training of the underlying base model is a significant consideration in the generated results. The models used for the evaluation include fine-tuned-codegen-6B-Verilog

with the base model `codegen-6B-multi`, `RTLCode-Deepseek-v1.1`, `OriGen` and `CodeV-DS-6.7B` with their base model `deepseek-coder-6.7b-base`, and our fine-tuned model with its base `Llama-3.1-8B-Instruct`.

Through these evaluations, we find that the fine-tuned models `VeriGen` and `CodeV` have a higher rate of copyright violations in comparison to their base models, with `VeriGen` increasing from 9% to 15%. This indicates that the fine-tuning process itself on the respective curated datasets plays a significant role in increasing the rates of generating copyright-protected hardware designs, with alternative methodologies and datasets potentially utilizing some of these copyright-protected codes in their fine-tuning dataset.

Furthermore, we can see that our fine-tuned model trained on hardware code without copyright protections, has the smallest hardware infringement rate (3%) out of all of the fine-tuned LLMs. Additionally, our model only depicts a 1% increase over its base model `Llama-3.1`. This further supports the notion that removing copyright-protected texts can significantly minimize the likelihood of generating these files in the inference process.

C. Evaluation of LLM Performance

After the continual pre-training is complete, the base `Llama-3.1-8B-Instruct` and our fine-tuned model `FreeV` are both evaluated on `VerilogEval` (Human) prompts through the methodology described in Section III-E2, and compared to the reported values from prior works [15], [33]. These include a set of foundational (pre-trained) models, along with state-of-the-art `Verilog-Tuned` models (Table II).

We can see that through continual pre-training on our curated dataset, `FreeV` demonstrates improvements in the ability to generate functional Verilog modules when compared to its base version. This is seen in improving the `pass@1`, `pass@5`, and `pass@10` rates by 0.7%, 7.9%, and 10.1% respectively. However, we can also see that the current training process does not provide state-of-the-art performance in comparison to prior works, as `FreeV` does not outperform other fine-tuning models. This is largely expected, as most other works implement additional fine-tuning beyond causal pre-training, including instruction-tuning for further alignment. Also, there may be limitations due to 4-bit quantization.

However, we highlight that relative improvements are seen through pre-training on `FreeSet` when compared to the base `Llama-3.1-8B-Instruct` model.

V. DISCUSSION AND FUTURE WORK

In this work, we seek to address the limitations of prior Verilog datasets with respect to copyright infringement of hardware designs. This is done through first establishing a preliminary evaluation benchmark that estimates how often fine-tuned LLMs generate a predefined set of copyright-protected hardware designs when prompted, with violations defined through a cosine-similarity threshold. Although this approach is feasible and easily implemented, future work can

explore additional prompting methods to target certain texts the LLM may have learned through fine-tuning. Furthermore, the dataset of copyrighted texts can be further expanded tests through additional curation efforts to increase the robustness of the final determination. Lastly, other similarity metrics may be explored for effective comparisons of the hardware design, such as evaluating the design structure itself like `GNN4IP` [35].

The work then proposes an automated dataset curation framework, resulting in the largest open-source dataset of Verilog files, while additionally providing filters for copyright-protected codes. Future works may look to expand the current real-world texts through data-augmentation techniques, or through providing natural-language descriptions of the files for instruction-tuning purposes.

Lastly, we demonstrate an example fine-tuning procedure through continual pre-training on an base `Llama` model. Although this procedure successfully brought relative improvements over its baseline, future works may implement more extensive training parameters (epochs, batch size, larger `LoRA` adapters) to extract additional functionality. Furthermore, methodologies may adapt this dataset to their alternative training strategies (such as instruction-tuning or reinforcement learning), enabling their limitations to be tested.

VI. CONCLUSION

Large language models have shown promise in their application to the hardware domain field, specifically in the generation of RTL code. Recent works have demonstrated success in applying emerging LLM fine-tuning techniques to LLMs for the task of RTL generation, enabling state-of-the-art performance on RTL tasks. However, these works remain limited by the dataset size of the hardware designs used in training, along with minimal checks regarding copyright infringement requirements for the curated files, resulting in the risk of fine-tuned LLMs violating copyright protections.

To address this limitation, we first propose a preliminary benchmark formulation to estimate the risk that existing LLM trained on Verilog codes will generate copyright-protected texts to a certain similarity threshold, indicating the use of protected hardware designs in the dataset. In an effort to then minimize this risk, we propose an end-to-end automated framework for extracting large quantities of Verilog files from online sources like `GitHub`, while notably also filtering out any licensed or copyright-protected files. This framework results in a curated and open-source dataset `FreeV` for Verilog codes resulting in 16.5 GB and over 222k files. This dataset was then utilized to fine-tune an 8B parameter `Llama` model through continual pre-training for hardware code generation.

Our evaluations indicate that our model `FreeV` demonstrates the lowest frequency of copyright-infringement (only 3% of codes), while also providing Verilog functionality improvements over its base model, increasing `VerilogEval` `pass@10` rates by over 10%.

ACKNOWLEDGMENT

The authors acknowledge the support from the Center for Secure Microelectronics Ecosystem – `CSME#210205`.

REFERENCES

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," 2023.
- [2] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz *et al.*, "Transformers: State-of-the-art natural language processing," in *Proceedings of the 2020 conference on empirical methods in natural language processing: system demonstrations*, 2020, pp. 38–45.
- [3] Tim Keary, "12 Practical Large Language Model (LLM) Applications," <https://www.techopedia.com/12-practical-large-language-model-llm-applications>, 2023, [Online; last accessed 21-Nov-2023].
- [4] D. Guo, Q. Zhu, D. Yang, Z. Xie, K. Dong, W. Zhang, G. Chen, X. Bi, Y. Wu, Y. Li *et al.*, "Deepseek-coder: When the large language model meets programming—the rise of code intelligence," *arXiv preprint arXiv:2401.14196*, 2024.
- [5] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. D. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.
- [6] Q. Zhu, D. Guo, Z. Shao, D. Yang, P. Wang, R. Xu, Y. Wu, Y. Li, H. Gao, S. Ma *et al.*, "Deepseek-coder-v2: Breaking the barrier of closed-source models in code intelligence," *arXiv preprint arXiv:2406.11931*, 2024.
- [7] D. Kocetkov, R. Li, L. B. Allal, J. Li, C. Mou, C. M. Ferrandis, Y. Jernite, M. Mitchell, S. Hughes, T. Wolf *et al.*, "The stack: 3 tb of permissively licensed source code," *arXiv preprint arXiv:2211.15533*, 2022.
- [8] J. Stratton, "An introduction to microsoft copilot," in *Copilot for Microsoft 365: Harness the Power of Generative AI in the Microsoft Apps You Use Every Day*. Springer, 2024, pp. 19–35.
- [9] B. Roziere, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, T. Remez, J. Rapin *et al.*, "Code llama: Open foundation models for code," *arXiv preprint arXiv:2308.12950*, 2023.
- [10] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat *et al.*, "Gpt-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.
- [11] R. Zhong, X. Du, S. Kai, Z. Tang, S. Xu, H.-L. Zhen, J. Hao, Q. Xu, M. Yuan, and J. Yan, "Llm4eda: Emerging progress in large language models for electronic design automation," *arXiv preprint arXiv:2401.12224*, 2023.
- [12] S. Thakur, B. Ahmad, H. Pearce, B. Tan, B. Dolan-Gavitt, R. Karri, and S. Garg, "Verigen: A large language model for verilog code generation," *ACM Transactions on Design Automation of Electronic Systems*, vol. 29, no. 3, pp. 1–31, 2024.
- [13] J. Blocklove, S. Garg, R. Karri, and H. Pearce, "Chip-chat: Challenges and opportunities in conversational hardware design," in *2023 ACM/IEEE 5th Workshop on Machine Learning for CAD (MLCAD)*. IEEE, 2023, pp. 1–6.
- [14] K. Chang, Y. Wang, H. Ren, M. Wang, S. Liang, Y. Han, H. Li, and X. Li, "Chipppt: How far are we from natural language hardware design," *arXiv preprint arXiv:2305.14019*, 2023.
- [15] Y. Zhao, D. Huang, C. Li, P. Jin, Z. Nan, T. Ma, L. Qi, Y. Pan, Z. Zhang, R. Zhang *et al.*, "Codev: Empowering llms for verilog generation through multi-level summarization," *arXiv preprint arXiv:2407.10424*.
- [16] S. Liu, W. Fang, Y. Lu, Q. Zhang, H. Zhang, and Z. Xie, "Rtlcoder: Outperforming gpt-3.5 in design rtl generation with our open-source dataset and lightweight solution," in *2024 IEEE LLM Aided Design Workshop (LAD)*. IEEE, 2024, pp. 1–5.
- [17] Z. Pei, H.-L. Zhen, M. Yuan, Y. Huang, and B. Yu, "Betternv: Controlled verilog generation with discriminative guidance," *arXiv preprint arXiv:2402.03375*, 2024.
- [18] N. Wang, B. Yao, J. Zhou, X. Wang, Z. Jiang, and N. Guan, "Large language model for verilog generation with golden code feedback," *arXiv preprint arXiv:2407.18271*, 2024.
- [19] S. Thakur, J. Blocklove, H. Pearce, B. Tan, S. Garg, and R. Karri, "Autochip: Automating hdl generation using llm feedback," *arXiv preprint arXiv:2311.04887*, 2023.
- [20] Y. Tsai, M. Liu, and H. Ren, "Rtlfixer: Automatically fixing rtl syntax errors with large language model," in *Proceedings of the 61st ACM/IEEE Design Automation Conference*, 2024, pp. 1–6.
- [21] B. Brittain, "Openai, microsoft want court to toss lawsuit accusing them of abusing open-source code," *Reuters*, Jan. 2023. [Online]. Available: <https://www.reuters.com/legal/litigation/openai-microsoft-want-court-toss-lawsuit-accusing-them-abusing-open-source-code-2023-01-27/>
- [22] R. S. Chakraborty and S. Bhunia, "Rtl hardware ip protection using key-based control and data flow obfuscation," in *2010 23rd International Conference on VLSI Design*, 2010, pp. 405–410.
- [23] M. Meeus, I. Shilov, M. Faysse, and Y.-A. de Montjoye, "Copyright traps for large language models," *arXiv preprint arXiv:2402.09363*, 2024.
- [24] N. Rahman and E. Santacana, "Beyond fair use: Legal risk evaluation for training llms on copyrighted text," in *ICML Workshop on Generative AI and Law*, 2023.
- [25] P. Samuelson, "Ongoing lawsuits could affect everyone who uses generative ai," *Science*, vol. 381, p. 6654, 2023.
- [26] J. Freeman, C. Rippe, E. DeBenedetti, and M. Andriushchenko, "Exploring memorization and copyright violation in frontier llms: A study of the new york times v. openai 2023 lawsuit," in *Neurips Safe Generative AI Workshop 2024*.
- [27] H. Li, G. Deng, Y. Liu, K. Wang, Y. Li, T. Zhang, Y. Liu, G. Xu, G. Xu, and H. Wang, "Digger: Detecting copyright content mis-usage in large language model training," *arXiv preprint arXiv:2401.00676*, 2024.
- [28] X. Liu, T. Sun, T. Xu, F. Wu, C. Wang, X. Wang, and J. Gao, "Shield: Evaluation and defense strategies for copyright compliance in llm text generation," *arXiv preprint arXiv:2406.12975*, 2024.
- [29] Y. Lu, S. Liu, Q. Zhang, and Z. Xie, "Rtlm: An open-source benchmark for design rtl generation with large language model," in *2024 29th Asia and South Pacific Design Automation Conference (ASP-DAC)*. IEEE, 2024, pp. 722–727.
- [30] R. Kande, H. Pearce, B. Tan, B. Dolan-Gavitt, S. Thakur, R. Karri, and J. Rajendran, "Llm-assisted generation of hardware assertions," *arXiv preprint arXiv:2306.14027*, 2023.
- [31] S. Liu, Y. Lu, W. Fang, M. Li, and Z. Xie, "Openllm-rtl: Open dataset and benchmark for llm-aided design rtl generation," 2024.
- [32] M. DeLorenzo, A. B. Chowdhury, V. Gohil, S. Thakur, R. Karri, S. Garg, and J. Rajendran, "Make every move count: Llm-based high-quality rtl code generation using mcts," *arXiv preprint arXiv:2402.03289*, 2024.
- [33] M. Liu, Y.-D. Tsai, W. Zhou, and H. Ren, "Craftrtl: High-quality synthetic data generation for verilog code models with correct-by-construction non-textual representations and targeted code repair," *arXiv preprint arXiv:2409.12993*, 2024.
- [34] F. Cui, C. Yin, K. Zhou, Y. Xiao, G. Sun, Q. Xu, Q. Guo, D. Song, D. Lin, X. Zhang *et al.*, "Origen: Enhancing rtl code generation with code-to-code augmentation and self-reflection," *arXiv preprint arXiv:2407.16237*, 2024.
- [35] R. Yasaei, S.-Y. Yu, E. K. Naeini, and M. A. Al Faruque, "Gnn4ip: Graph neural network for hardware intellectual property piracy detection," in *2021 58th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2021, pp. 217–222.